ECE3232/ME4673

PCU API Documentation

W2026

Jan 21 2026 (Rev 1.0)

CONTENTS

1. INTRODUCTION 1

2. TROUBLESHOOTING 2

2.1. Important Implementation Notes 2
2.2. Common UART Debugging Checklist 2
2.2.1. Physical Layer (check this first, always) 2
2.2.2. UART Configuration 2
2.2.3. Logic Analyzer / Serial Monitor (Highly Recommended) 3
2.2.4. Receive Path (MOST COMMON FAILURE) 3
2.2.5. Message Framing 3
2.2.6. Payload Handling 4
2.2.7. Transmit Path 4
2.2.8. Expected Responses 4
2.2.9. Known Failure Symptoms 4
2.2.10. Final Sanity Checks 4

3. MESSAGE PROTOCOL 5

3.1. Message Format 5
3.2. Message IDs 6
3.3. Message Contents 7
3.3.1. Message 0300h: Unknown/Invalid Message ID 7
3.3.2. Message 0401h: Get PCU Info Command 8
3.3.3. Message 0402h: Get PCU Info Response 9
3.3.4. Message 0501h: Get User Data Command 10
3.3.5. Message 0502h: Get User Data Response 11
3.3.6. Message 0601h: Set Motor Settings Command 12
3.3.7. Message 0701h: Set Servo Pulses Command 13
3.3.8. Message 0801h: Set Laser Scope Command 14
3.3.9. Message 0901h: Shoot Laser (Damage) Command 15
3.3.10. Message 0902h: Shoot Laser (Turret Shield Code) Command 16
3.3.11. Message 0903h: Shoot Laser (Request Repair Code) Command 17
3.3.12. Message 0904h: Shoot Laser (Transmit Repair Code) Command 18
3.3.13. Message 0A01h: Surface Exploration Command 19
3.3.14. Message 0A03h: Processing Plant Ore Type Command 20

1. INTRODUCTION

This document defines the **UART-based application programming interface (API)** used to communicate between a student-controlled system and the **Peripheral Control Unit (PCU)** for *The Harvest* competition.

Students in **ECE3232** will communicate with the PCU using an **MPLAB Xpress development board**, while students in **ME4673** will use a **UNB Development Board**. Regardless of platform, **all communication with the PCU must occur over UART**, using the designated **HCNS click boards** and a **USB-C cable** connection.

The PCU operates as a command-driven device. Students are required to write embedded software that:

- Constructs and transmits valid UART messages exactly as defined in this document
- Correctly interprets (parses) responses returned by the PCU
- Handles message framing, payload sizing, and message IDs reliably

This document specifies:

- The UART message protocol and framing structure
- All supported message IDs and their associated payload formats
- The direction of communication (User → PCU or PCU → User)
- Expected responses, where applicable

Strict adherence to this API is required for reliable operation during gameplay. Messages that do not follow the defined protocol may be ignored or explicitly rejected by the PCU.

2. TROUBLESHOOTING

When in doubt, **start here**. Use this section to troubleshoot your design before reaching out for help. You'll learn the most when you take a few minutes to investigate the issue and understand what's happening.

2.1. Important Implementation Notes

- **Sync bytes matter.** Every message **must** begin with the two sync bytes `0xFE 0x19`. If these are incorrect or missing, the PCU will not recognize the message.
- **Byte order matters.** Multi-byte fields (Message ID, payload size, and multi-byte values) are transmitted **LSB first, then MSB** unless explicitly stated otherwise.
- **Payload size must be exact.** The payload size fields must match the number of payload bytes sent. Incorrect lengths will cause messages to be rejected or misinterpreted.
- **UART is a byte stream, not a message buffer.** Bytes arrive one at a time and must be processed as they are received. Do **not** assume complete messages are stored automatically.
- **ECE3232 (MPLAB Xpress users):**
  The UART receive buffer is **only one byte deep**. If incoming bytes are not read immediately (typically using an interrupt-driven receiver), the buffer will be overwritten, an overflow flag will be set, and **no further receive interrupts will occur** until the error is cleared.
- **ME4673 (UNB Development Board users):**
  UART data is **not automatically retained**. If the provided UART receive functions are not used, incoming data will be lost. Your code must explicitly handle received bytes using the supported API.
- **Do not assume a response.** Some commands generate a response from the PCU; many do not. Your software must handle both cases safely.

2.2. Common UART Debugging Checklist

Before asking for help, ensure you've checked off all of these items (where applicable and where you're capable).

2.2.1. Physical Layer (check this first, always)

- ☐ **HCNS click board** is used on both the Xpress/UNB Dev and the PCU?
- ☐ USB-C cable is **data-capable** (if not using provided cable)
- ☐ No loose headers or rotated click boards

2.2.2. UART Configuration

- ☐ Baud rate = **115200 bps**
- ☐ **8 data bits, no parity, 1 stop bit (8-N-1)**
- ☐ UART peripheral clock source is correct (ECE3232)
- ☐ UART enabled *before* sending or receiving data

A wrong clock source can give you “almost correct” garbage: the worst kind.

2.2.3. Logic Analyzer / Serial Monitor (Highly Recommended)

- ☐ Confirm **actual bytes on the wire**
- ☐ Verify sync bytes appear correctly
- ☐ Confirm payload length matches transmitted bytes
- ☐ Check baud rate using analyzer auto-detect

If software says one thing and the wire says another, **believe the wire**.

2.2.4. Receive Path (MOST COMMON FAILURE)

- ☐ You are **not** assuming that the UART stores messages
- ☐ Each **received byte is read immediately**
- ☐ **ECE3232 (MPLAB Xpress):**
  - ☐ RX handled via **interrupt**
  - ☐ Overrun error flag is checked (should never be set)
  - ☐ RX register is read every interrupt
- ☐ **ME4673 (UNB Dev Board):**
  - ☐ Using the **provided UART receive functions**
  - ☐ No custom polling that blocks execution

If bytes arrive faster than you read them, **data is lost silently**.

2.2.5. Message Framing

- ☐ Sync bytes are **exactly** `0xFE 0x19`
- ☐ You are searching for sync bytes in the stream (not assuming alignment)
- ☐ Message ID read as **LSB first**
- ☐ Payload size read as **LSB first**
- ☐ You wait until **all bytes** arrive before parsing payload

UART has no concept of “message boundaries” — *you* create them.

2.2.6. Payload Handling

- ☐ Payload size matches the message definition exactly
- ☐ No extra bytes are sent
- ☐ No missing bytes are skipped
- ☐ Multi-byte values reconstructed correctly (try test scenarios/debugger)

One wrong payload length will desynchronize *every* message after it.

2.2.7. Transmit Path

- ☐ Transmit buffer is empty before writing next byte
- ☐ Entire message is sent in order (header → payload)
- ☐ No blocking delays between bytes (unless explicitly required)
- ☐ Data type is `uint8_t` (ECE3232)

2.2.8. Expected Responses

- ☐ You only wait for responses when the API says one exists
- ☐ Your code does **not block indefinitely** waiting for a reply
- ☐ Timeouts are handled safely

Many commands are "fire and forget."

2.2.9. Known Failure Symptoms

Symptom	Likely Cause
No RX interrupts after first message	UART overrun flag set
Random message IDs	Byte order or framing error
Works once, then stops	RX buffer overflow
Payload values wrong	LSB/MSB swapped
Nothing happens at all	TX/RX swapped or wrong baud

2.2.10. Final Sanity Checks

- ☐ Hard reset both boards
- ☐ Re-send a **known-good command** (e.g. `0401h Get PCU Info`)
- ☐ Reduce code to **minimum RX + print bytes**
- ☐ Add LED toggle inside RX interrupt to confirm activity

3. MESSAGE PROTOCOL

The message protocol used by the user to communicate with their PCU unit is a request-acknowledge format. Some messages received will be acknowledged with a valid message response while others will simply not respond. The default baudrate used is 115200bps.

3.1. Message Format

Messages are of variable length and observe the following Header-Payload structure:

1	2	3	4	5	6	7	8	6 + n
Sync #1	Sync #2	Msg ID LSB	Msg ID MSB	Payload Size LSB	Payload Size MSB	Payload (Byte 0)	Payload (Byte 1)	Payload (Byte n)

Header

Bytes 1 & 2:

The first two bytes of the message header are Sync Bytes used to determine the start of a message. The values for sync #1 and #2 are 0xFE and 0x19 respectively.

Bytes 3 & 4 :

The next two bytes of the header are the message ID which defines the type of message being sent. This message ID defines the format for the payload to follow.

Bytes 5 & 6:

The fifth and sixth bytes contain the payload size in number of bytes. The payload size may be zero for certain message types indicating that all required information is present in the message header.

Payload

The payload portion of the message is optional and contains data that is specific to the message ID sent in the header.

3.2. Message IDs

The following list includes all supported message IDs, their name, the payload size, the sender and the expected response:

ID	Message	Payload Size	Sender	Expected Response
0300h	Unknown/Invalid Message ID	0	PCU	N/A
0401h	Get PCU Info Command	0	User	0402h
0402h	Get PCU Info Response	6	PCU	N/A
0403h	Set PCU Info Command	3	User	N/A
0501h	Get User Data Command	0	User	0502h
0502h	Get User Data Response	20	PCU	N/A
0601h	Set Motor Settings Command	4	User	N/A
0701h	Set Servo Pulses Command	3	User	N/A
0801h	Set Laser Scope Command	1	User	N/A
0901h	Shoot Laser Command	1	User	N/A
0902h	Transmit Shield Code Laser Command	0	User	N/A
0903h	Request Repair Code Laser Command	0	User	N/A
0904h	Transmit Repair Code Laser Command	0	User	N/A
0A01h	Surface Exploration Command	4	User	N/A
0A03h	Processing Plant Ore Type Command	1	User	N/A

3.3. Message Contents

3.3.1. Message 0300h: Unknown/Invalid Message ID

Description: This message is sent by the PCU to the user's controller to specify that an unknown or invalid message ID was received. It is used as a generic NOT acknowledge in response to any unknown command from the user.

Sender: PCU

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**00h**	**03h**	**00h**	**00h**

Expected Message Response: N/A

3.3.2. Message 0401h: Get PCU Info Command

Description: This message command is sent by the user to the PCU to get system information.

Sender: User

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**01h**	**04h**	**00h**	**00h**

Expected Message Response: 0402h – Get PCU Info Response

3.3.3. Message 0402h: Get PCU Info Response

Description: This message response is sent by the PCU to the user and returns the team ID, player ID, overall health, shield code flag and repair code flag values.

Sender: PCU

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**02h**	**04h**	**06h**	**00h**

7 Team ID	8 Player ID	9 Overall Health LSB	10 Overall Health MSB	11 Shield Code Flag	12 Repair Code Flag
##h	**##h**	**##h**	**##h**	**##h**	**##h**

Expected Message Response: N/A

Note: A shield code flag value of 0 will be returned if the user has not yet received a shield code from the laser turrets during gameplay. Similarly, a repair code flag value of 0 will be returned if the user has not yet received a repair code from the processing plant during gameplay.

3.3.4. Message 0501h: Get User Data Command

Description: This message is sent by the user to the PCU to get the most recent user data received from the wireless transmitter.

Sender: User

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**01h**	**05h**	**00h**	**00h**

Expected Message Response: 0502h – Get User Data Response

3.3.5. Message 0502h: Get User Data Response

Description: This message is sent by the PCU to the user and returns the current transmitter information.

Sender: PCU

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**02h**	**05h**	**14h**	**00h**

7 Right Joystick X LSB	8 Right Joystick X MSB	9 Right Joystick Y LSB	10 Right Joystick Y MSB	11 Left Joystick Y LSB
##h	**##h**	**##h**	**##h**	**##h**

12 Left Joystick Y MSB	13 Left Joystick X LSB	14 Left Joystick X MSB	15 Switch A LSB	16 Switch A MSB
##h	**##h**	**##h**	**##h**	**##h**

17 Switch B LSB	18 Switch B MSB	19 Switch C LSB	20 Switch C MSB	21 Switch D LSB
##h	**##h**	**##h**	**##h**	**##h**

22 Switch D MSB	23 Potentiometer VRA LSB	24 Potentiometer VRA MSB	25 Potentiometer VRB LSB	26 Potentiometer VRB MSB
##h	**##h**	**##h**	**##h**	**##h**

Expected Message Response: N/A

3.3.6. Message 0601h: Set Motor Settings Command

Description: This message is sent by the user to the PCU to configure the PWM characteristics of the motor controller.

Sender: User

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**01h**	**06h**	**04h**	**00h**

7 Motor A Direction	8 Motor A PWM	9 Motor B Direction	10 Motor B PWM
##h	**##h**	**##h**	**##h**

Expected Message Response: N/A

Motor Direction Values:

0 – Brake
1 – Forward
2 – Backward

Motor PWM Range:

0 → 100

3.3.7. Message 0701h: Set Servo Pulses Command

Description: This message is sent by the user to the PCU to set the digital servo pulse values.

Sender: User

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**01h**	**07h**	**03h**	**00h**

7 Servo 1 Pulse Value	8 Servo 2 Pulse Value	9 Servo 3 Pulse Value
##h	**##h**	**##h**

Expected Message Response: N/A

Servo Pulse Range (10s of microseconds):

$0 \rightarrow 250$

3.3.8. Message 0801h: Set Laser Scope Command

Description: This message is sent by the user to the PCU to specify whether to have the laser scope on or off.

Sender: User

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**01h**	**08h**	**01h**	**00h**

7 Enable Flag
##h

Expected Message Response: N/A

Laser Scope Enable Flag:

0 – Scope disabled
1 – Scope enabled

3.3.9. Message 0901h: Shoot Laser (Damage) Command

Description: This message is sent by the user to the PCU to specify the user's intent to shoot the laser. The payload byte specifies the type of 'ammunition' used by the tank to fire the shot.

Sender: User

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**01h**	**09h**	**01h**	**00h**

7 Shot Type
##h

Expected Message Response: N/A

Laser Shot Type:

1 – Low Caliber Shot
2 – High Caliber Shot

Note: For TheHarvest competition, the shot type will not have any effect on the actual gameplay (they will all be considered the same amount of damage, etc.).

3.3.10. Message 0902h: Shoot Laser (Turret Shield Code) Command

Description: This message is sent by the user to the PCU to specify the user's intent to shoot the turret shield code.

Sender: User

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**02h**	**09h**	**00h**	**00h**

Expected Message Response: N/A

3.3.11. Message 0903h: Shoot Laser (Request Repair Code) Command

Description: This message is sent by the user to the PCU to specify the user's desire to receive a repair code from the processing plant. The processing plant will wirelessly send the code to the user.

Sender: User

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**03h**	**09h**	**00h**	**00h**

Expected Message Response: N/A

3.3.12. Message 0904h: Shoot Laser (Transmit Repair Code) Command

Description: This message is sent by the user to the PCU to specify the user's intent to shoot the specified Repair code.

Sender: User

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**04h**	**09h**	**00h**	**00h**

Expected Message Response: N/A

3.3.13. Message 0A01h: Surface Exploration Command

Description: This message is sent by the User to the PCU with the results obtained while performing a task during surface exploration.

Sender: User

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**01h**	**0Ah**	**04h**	**00h**

7 Task ID LSB	8 Task ID MSB	9 Task-Specific Value LSB	10 Task-Specific Value MSB
##h	**##h**	**##h**	**##h**

Expected Message Response: N/A

Task ID:

1 – RFID Tag ID
2 – Fundamental Frequency ID
3 – Magnetic Area ID
4 – Conductive Area ID
5 – IR Source Area ID

Task-Specific Value:

RFID Tag ID: 2 LSBs of RFID Tag value obtained from RFID reader
Fundamental Frequency ID: Value of Fundamental Frequency calculated using FFT
Magnetic Area ID: Zone number where a magnetic field was detected
Conductive Area ID: Zone number where conductive area was the highest
IR Source Area ID: Zone number where an IR source was detected

3.3.14. Message 0A03h: Processing Plant Ore Type Command

Description: This message is sent by the user to the PCU to specify the desired ore type for processing. The PCU will then transmit a laser message with this information intended for the processing plant. The payload byte specifies the type of ore planned for processing.

Sender: User

Format:

1 Sync #1	2 Sync #2	3 Msg ID LSB	4 Msg ID MSB	5 Payload Size LSB	6 Payload Size MSB
FEh	**19h**	**03h**	**0Ah**	**01h**	**00h**

7 Ore Type
##h

Expected Message Response: N/A

Ore Type:

1 – SolarFlare (Red)
2 – Aurorium (Yellow)
3 – Cobaltite (Blue)
4 – Solarium (Red/Yellow)
5 – Aurotite (Blue/Yellow)